

Overview

Mobile Build Blueprint — webapp-main

Code-only mobile implementation draft

Snapshot cutoff: Jul 10, 2026 at 18:00 IST (12:30 UTC).

Resolved source

Repository: kashyapempiricinfotech-sys/webapp-main • branch: main

Exact cutoff commit: 368cec4aff9f293a2564eb6c35f3d9045b343c2c

The commit is the merge base of the current main comparison; main is two commits ahead.

The later experimental scanner and legacy-cookie hardening commits are outside this snapshot and are intentionally excluded. Evidence: repository commit comparison and README.md:1–13.

Method: static reading of all 34 files at the resolved commit, including api, server/routes, app, src, package.json, and packages/data/prisma/schema.prisma. No app process was started and no application endpoint was contacted.

Decision in brief

Recommendation (inferred): build a separate React Native app with Expo development builds, retain the Express API as a hardened service boundary, share validated TypeScript contracts rather than DOM components, and close the auth, settings, push-installation, payment-return, and persistence gaps before release.

Tab guide

- Architecture — runtime topology, modules, complete declared endpoint/auth inventory, integrations, and inactive or mismatched paths.
- Screen Inventory — all six cutoff screens and every reusable UI implementation, alias, and wrapper with observed reuse counts.
- Mobile Stack — framework, bridge, testing, and CI/CD alternatives, recommendation, current published tiers, and a rough cost scenario.
- User Flow — buildable screen-to-screen journey, branches, and explicit inferred navigation where the fixture does not wire it.
- Data Model & Contracts — Prisma entities, TypeScript DTOs, endpoint payloads, and client/server contract gaps.
- Delivery Plan — four dependency-ordered phases across Sprint 1 through Sprint 6, acceptance gates, and Jira mapping.
- Risks & Open Questions — blockers, decisions, and evidence needed before implementation or store release.
- Secrets found (review) — credential-shaped locations and remediation actions only; no values are reproduced.

Reading convention

A path and line reference grounds each code finding in the cutoff tree. “(inferred)” marks proposed behavior, interpretation, or navigation not directly implemented by the fixture. Vendor cost facts are linked to current published sources on Mobile Stack. This is a draft plan, not a deployment authorization.

Architecture

Architecture

Cutoff runtime, business modules, APIs, authentication, and integrations

Runtime topology

The manifest declares a private TypeScript project with Next 15.4, React 19.1, Express 5.1, jsonwebtoken, cookie-session, Stripe, posthog-js, @sentry/node, tsx, TypeScript, and Vitest; scripts separately invoke Next development, the Express server, typecheck, and Vitest.

Source: package.json:1–27.

Next/React page modules live under app. Express creates JSON middleware, starts analytics and conditional error reporting, mounts routers, expands a route manifest, and listens on port 3001. The repository also contains a public GET handler at api/health.ts.

Sources: server/index.ts:1–26; api/health.ts:1–3; README.md:5–11.

The cutoff tree contains no visible Next proxy/rewrite, deployment configuration, lockfile, API base-URL configuration, or automated test file among the 34 committed files (inferred from the complete cutoff file list). The relative /api client calls therefore require a deployment/proxy contract to be settled for mobile; do not assume port 3001 is reachable from a device (inferred).

Business logic and boundaries

- Authentication: src/services/auth.ts:3–11 posts login and constructs Bearer headers; server/routes/auth.ts:5–12 issues and refreshes fixture tokens; server/middleware/requireAuth.ts:4–13 verifies Bearer tokens.
- Invoices and checkout: src/services/invoices.ts:4–13 lists invoices and requests checkout; server/routes/invoices.ts:6–10 returns an empty list and creates a Stripe Checkout session through src/integrations/stripe.ts:3–12.
- Notification preferences: server/routes/notifications.ts:3–5 exposes authenticated get/put preference handlers. The push-center page is feature-gated in app/notifications-center/page.tsx:3–6. The client registration service targets a different installation path in src/services/notifications.ts:5–7.
- Settings: app/settings/page.tsx:2–7 calls src/services/settings.ts:3–5, which targets PUT /api/settings. No matching server route is mounted in server/index.ts:16–24 or declared in server/routes/manifest.ts:5–8 (inferred contract gap).
- Admin reporting: app/admin-reports/page.tsx:4–7 renders a forbidden branch unless the supplied user role is admin. server/routes/manifest.ts:7 declares authenticated audit events, but no revenue-report API is declared; src/services/reportingAggregator.ts:3–6 is explicitly unwired dead code. Do not describe that aggregator as active.
- Persistence model: packages/data/prisma/schema.prisma:1–32 describes PostgreSQL User, Invoice, and NotificationSetting relationships. The visible server handlers return fixture data rather than querying this schema, and package.json does not list Prisma runtime/client dependencies (inferred implementation gap).

Authentication and authorization flows

Current path: login is public; the handler derives admin/user from an email suffix, signs a fixed subject, returns token plus user, and does not inspect the submitted password in the visible handler. Refresh returns a fixture token without an auth guard in its router. Protected invoice, notification, and authenticated manifest routes use requireAuth. Sources: server/routes/auth.ts:5–12; server/index.ts:16–24; server/middleware/requireAuth.ts:4–13. Legacy path: /api/legacy/profile and /api/legacy/logout remain mounted and use cookie-session middleware. At the cutoff, the cookie is HTTP-only and has an environment fallback; the later hardening commit must not be backported into this analysis. Sources: server/index.ts:19; server/routes/legacy.ts:3–7; server/auth/legacySession.ts:3–8. Mobile target (inferred): use short-lived access tokens in platform secure storage, a server-defined rotating refresh/revocation contract, expiry/audience/issuer/algorithm validation, server-side role authorization, logout invalidation, and separate development/staging/production configuration. Do not migrate mobile onto deprecated cookie endpoints without an explicit compatibility decision.

Third-party integrations

- Stripe is active on the authenticated invoice checkout route. The server constructs Checkout with fixed USD amount and custom-scheme success/cancel return URLs. Sources: server/routes/invoices.ts:6–10; src/integrations/stripe.ts:3–12.
- PostHog startup is called during Express initialization; the integration imports the browser posthog-js package and defines an invoice-open event helper. The helper has no visible caller in the cutoff tree (inferred). Sources: server/index.ts:2,12; src/integrations/posthog.ts:1–9.
- Sentry initialization is wired at server startup but returns false unless its environment flag is true. The example environment defaults it off. Sources: server/index.ts:3,13; src/integrations/sentry.ts:3–7; .env.example:4.
- Push UI is disabled unless its feature flag is true; notification preferences are still registered server-side. The client installation registration route has no corresponding mounted handler in the cutoff tree (inferred). Sources: app/notifications-center/page.tsx:3–6; server/index.ts:18; server/routes/notifications.ts:3–5; src/services/notifications.ts:5–7.

Declared endpoint inventory

The native table below distinguishes server-declared routes from client-only calls. Auth labels follow actual mounts and manifest flags, not route names.

Method & path	Auth / status	Behavior and cutoff evidence
GET /api/health	Public handler	Returns {ok:true}; api/health.ts:1–3. Confirm deployment routing (inferred).
POST /api/auth/login	Public	Signs fixture token and returns user; role derives from email suffix; server/routes/auth.ts:5–9.

Method & path	Auth / status	Behavior and cutoff evidence
POST /api/auth/refresh	Public in visible mount	Returns fixed fixture-token-shaped response; server/routes/auth.ts:12; server/index.ts:16.
GET /api/invoices	Bearer JWT	Returns empty list; server/index.ts:17; server/routes/invoices.ts:6.
POST /api/invoices/:invoiceId/checkout	Bearer JWT	Creates Stripe Checkout session and returns URL; server/index.ts:17; server/routes/invoices.ts:7–10.
GET /api/notifications/preferences	Bearer JWT	Returns paymentReminders true; server/index.ts:18; server/routes/notifications.ts:3–4.
PUT /api/notifications/preferences	Bearer JWT	Coerces request value to Boolean; server/index.ts:18; server/routes/notifications.ts:5.
GET /api/legacy/profile	Cookie session; deprecated/live	Returns session user id or null; server/index.ts:19; server/routes/legacy.ts:3–6.
POST /api/legacy/logout	Cookie session; deprecated/live	Clears session and returns 204; server/routes/legacy.ts:7.
GET /api/catalog	Public manifest entry	Returns starter catalog item; server/index.ts:21–24; server/routes/manifest.ts:5–6.
GET /api/audit-events	Bearer JWT manifest entry	Returns empty array; server/index.ts:21–24; server/routes/manifest.ts:7.
PUT /api/settings	Client-only; no matching server mount	Called by settings service; src/services/settings.ts:3–5. Contract gap (inferred).
POST /api/notifications/installations	Client-only; no matching server mount	Called by push registration service; src/services/notifications.ts:5–7. Contract gap (inferred).

Screen Inventory

Screen Inventory

Complete cutoff UI inventory and observed reuse

Count rule: a screen counts once per page module; reusable-component frequency counts distinct cutoff page consumers, resolving the barrel alias and HOC composition. It is a static reference count, not runtime telemetry.

Screens

The first native table lists every `app/*/page.tsx` in the cutoff tree. No root, invoice-detail, checkout-result, catalog, audit-event, or legacy-profile page module is present in the 34-file snapshot (inferred from the complete file list).

Screen / route	Purpose and state	Observed reuse / source
Login /login	Sign-in section and Continue CTA; client component invokes <code>signIn</code> with demo arguments.	One page module; <code>app/login/page.tsx:1–7</code> .
Dashboard /dashboard	Account snapshot and Review invoices CTA; navigation handler is not visible (inferred).	One page module; <code>app/dashboard/page.tsx:1–5</code> .
Invoices /invoices	Demo InvoiceCard and telemetry-wrapped Create invoice CTA; service layer separately lists and checks out.	One page module; <code>app/invoices/page.tsx:1–10</code> ; <code>src/services/invoices.ts:4–13</code> .
Settings /settings	Save payment-reminder preference via settings service; contract mismatch noted.	One page module; <code>app/settings/page.tsx:1–7</code> .
Notifications Center /notifications-center	Payment reminder preferences; returns null unless <code>FEATURE_PUSH</code> is true.	One page module; <code>app/notifications-center/page.tsx:1–6</code> .
Admin Reports /admin-reports	Restricted revenue-report placeholder; non-admin sees Forbidden.	One page module; <code>app/admin-reports/page.tsx:1–7</code> .

Reusable components

The second native table resolves `PrimaryCta` to `ActionButton` and counts the telemetry-wrapped use on Invoices. Source: `src/components/index.ts:1–3`; app page imports.

Reusable UI	Purpose	Distinct observed page consumers
ActionButton → PrimaryCta alias	Generic button with optional onClick; re-exported under a semantic alias.	4: Login, Dashboard, Invoices (through withTelemetry), Settings. src/components/ActionButton.tsx:3–5 ; index.ts:1.
AppShell	main/header/title wrapper around children.	5: Dashboard, Invoices, Settings, Notifications Center, Admin Reports. src/components/AppShell.tsx:3–5.
InvoiceCard	Displays invoice number and status.	1: Invoices. src/components/InvoiceCard.tsx:3–5.
withTelemetry HOC	Wraps a component in analytics-marked div.	1 composition site: Invoices. src/components/withTelemetry.tsx:3–7; app/invoices/page.tsx:2–5.
TrackedCreateButton (derived)	Page-local telemetry-wrapped PrimaryCta, not a standalone exported module.	1 render on Invoices; app/invoices/page.tsx:5,8–10.

Mobile Stack

Mobile Stack

Decision record, alternatives, published tiers, and rough cost

Pricing checked: Aug 4, 2026 . USD amounts below are rough planning figures from the linked vendor pages, before tax, overages, payment processing, backend hosting, analytics volume, and team labor. Annual-billed equivalents are identified.

Recommended architecture

Recommendation (inferred): React Native with Expo development builds and Expo Router; a typed API client with an environment-specific HTTPS base URL; TanStack Query-style server-state caching (option to validate during foundation); platform secure storage for tokens; native deep linking for checkout returns; Expo/native notification and analytics adapters behind interfaces; and an independently hardened Express API. Share domain types and contract tests, not the existing HTML/DOM components.

The choice follows the existing React/TypeScript page and service shape (package.json:10–25; src/types.ts:1–16; src/services/auth.ts:3–11) while replacing web-only markup such as main, header, button, article, and section (src/components/ActionButton.tsx:3–5; AppShell.tsx:3–5; InvoiceCard.tsx:3–5; app/login/page.tsx:5–7).

Options and decision

The native comparison table below gives at least two real options in each requested category. “Open-source/self-hosted” is the priced tier where no vendor subscription is required; infrastructure and labor are separate.

Category	Option / priced tier / rough monthly cost	Trade-off and disposition
Framework	React Native + Expo — Free tier, \$0/month for framework/limited EAS.	Recommended (inferred): retains React/TypeScript skills and DTO/service concepts; HTML components must be rebuilt with native primitives. Paid CI is priced separately. Expo pricing.
Framework	Flutter — free/open-source tier, \$0/month license.	Strong compiled, consistent UI and plugin ecosystem; requires Dart rewrite of UI and client logic/types. Alternative (inferred). Flutter FAQ.
Native bridge	Expo Modules/SDK — Free/open-source development tier, \$0/month license.	Recommended (inferred): Swift/Kotlin module API, secure storage, linking, notification adapters and config plugins fit

Category	Option / priced tier / rough monthly cost	Trade-off and disposition
		an RN app; development builds needed for custom native code. Expo Modules API.
Native bridge	Capacitor — open-source/self-hosted tier, \$0/month license.	Web-native container and plugin API can retain web technology, but WebView/Next routing, SSR and mobile UX constraints require validation; weaker fit for a native-first rebuild (inferred). Capacitor overview.
Testing	Detox — MIT open-source/self-hosted tier, \$0/month license.	Recommended E2E layer (inferred): gray-box React Native iOS/Android automation on local/CI simulators; runner/device costs remain separate. Detox repository.
Testing	BrowserStack App Automate Device Cloud — 1 parallel, \$199/month equivalent billed annually.	Real-device coverage across iOS/Android; recurring annual commitment and one-parallel throughput trade-off. Add for release matrix (inferred). BrowserStack pricing.
CI/CD	Expo EAS Production — \$199/month plus usage.	Recommended (inferred): \$225 build credit, two included concurrencies, store submission/update workflow and 50K update MAUs on current pricing; overages apply. Expo pricing.
CI/CD	Codemagic Pay-as-you-go — \$0 base; illustrative \$117.50/month.	At 1,000 Mac mini M2 min × \$0.095 + 500 Linux min × \$0.045 = \$117.50; one included concurrency, extra concurrency \$49/month. Flexible alternative; actual minutes vary. Codemagic pricing.

Bridge scope for this app

- Secure storage and lifecycle: access/refresh credentials, logout, biometric unlock only if product approves (inferred).
- Deep links: register and validate the custom invoice checkout return scheme, bind result to server-side invoice state, and handle cold/warm starts and cancellation (inferred from `src/integrations/stripe.ts:8–11`).
- Push: permissions, APNs/FCM token lifecycle, installation registration, preference sync, revoked-token cleanup, and flag-controlled rollout (inferred from `app/notifications-center/page.tsx:3–6` and `src/services/notifications.ts:5–7`).
- Payments and observability: use platform-appropriate Stripe integration and privacy-consented mobile analytics/error adapters; validate app-store payment rules for the invoice use case before choosing in-app versus external checkout (inferred from `src/integrations/stripe.ts:3–12`, `posthog.ts:1–9`, and `sentry.ts:3–7`).

CI/CD target pipeline

Proposed gates (inferred): pull request lint/typecheck/unit and contract tests → Android/iOS development builds → simulator/emulator E2E smoke → signed staging artifacts and internal distribution → real-device regression, accessibility, deep-link, offline, permission, and upgrade tests → manual release approval → store submission. Keep signing credentials in the chosen provider's secret store, use environment-scoped API origins, retain build artifacts, and promote immutable versioned builds.

Budget scenario

Recommended planning scenario (inferred): EAS Production \$199/month plus BrowserStack Device Cloud \$199/month equivalent billed annually = approximately \$398/month before variable usage and taxes. Apple Developer Program is \$99/year (about \$8.25/month amortized); Google Play registration is \$25 one time. Framework, Expo Modules, and Detox license lines are \$0. Do not add the alternative Codemagic estimate to the recommended total.

Sources: Expo pricing; Flutter FAQ; Capacitor overview; Expo Modules API; Detox repository; BrowserStack pricing; Codemagic pricing; Apple Developer Program; Google Play Console registration.

User Flow

User Flow

Screen-to-screen implementation flow and missing links

The fixture contains page components and service calls but no visible navigation links/router transitions between most screens. The sequence below is a proposed mobile journey (inferred), grounded in page purpose, service endpoints, and checkout return URLs; it is not a claim of existing wired web navigation.

Primary customer journey

1. Launch → restore a valid secure session or show Login (inferred). The cutoff Login page invokes `signIn`; service posts `/api/auth/login`. Sources: `app/login/page.tsx:1–7`; `src/services/auth.ts:3–7`.
2. Login success → Dashboard (inferred). Store credentials securely, display account snapshot, and route failures to an inline error/retry state (inferred). Source: `app/dashboard/page.tsx:3–5`.
3. Dashboard “Review invoices” → Invoices (inferred; the button has no visible navigation handler). Source: `app/dashboard/page.tsx:3–5`.
4. Invoices → load authenticated list, show loading/empty/error states, select invoice, and offer checkout (inferred). The visible page uses demo data and a tracked create CTA; the service calls `GET /api/invoices` and `POST /api/invoices/:invoiceId/checkout`. Sources: `app/invoices/page.tsx:1–10`; `src/services/invoices.ts:4–13`.
5. Checkout → open the server-created Stripe session, then return through the custom scheme to an invoice-result/detail destination (inferred). Success and cancellation return URLs are constructed, but no cutoff detail/result page is present. Source: `src/integrations/stripe.ts:5–12`.
6. Settings → edit payment reminders and save (inferred). The visible client targets `PUT /api/settings`, while the mounted server preference path is `/api/notifications/preferences`; resolve this before building. Sources: `app/settings/page.tsx:1–7`; `src/services/settings.ts:3–5`; `server/routes/notifications.ts:3–5`.
7. Notifications Center → permission prompt, preferences, and installation registration only after the feature flag and server contract are ready (inferred). The page returns null when disabled, and the client registration path is not mounted. Sources: `app/notifications-center/page.tsx:3–6`; `src/services/notifications.ts:5–7`; `server/index.ts:16–24`.
8. Logout → revoke/clear mobile credentials and return to Login (inferred). The only visible logout endpoint is the legacy cookie route; do not assume it revokes JWTs. Source: `server/routes/legacy.ts:3–7`.

Role and error branches

- Admin user → Admin Reports (inferred). The page checks a supplied role and otherwise renders Forbidden; server-side authorization and a report data contract are still required. Source: `app/admin-reports/page.tsx:4–7`.
- Missing/invalid Bearer token → 401 → one bounded refresh attempt or re-authentication (inferred). Middleware returns distinct missing/invalid errors; refresh behavior is fixture-only. Sources: `server/middleware/requireAuth.ts:4–13`; `server/routes/auth.ts:12`.
- Offline/timeout → cached read-only content where safe, explicit retry, and no duplicate checkout mutation (inferred). No offline implementation is present in the cutoff tree.

Navigation map (inferred)

Launch → Login → Dashboard → Invoices → Checkout → Invoice result/detail → Invoices; Dashboard ↔ Settings; Settings ↔ Notifications Center (flagged); Dashboard → Admin Reports (admin only); any authenticated screen → re-authentication on unrecoverable 401; Logout → Login.

Data Model & Contracts

Data Model & Contracts

Persistence shape, DTOs, and API contract decisions

Persistence model

Prisma declares a PostgreSQL datasource and three models. User has a unique email and role, one-to-many invoices, and an optional one-to-one notification setting. Invoice belongs to User through ownerId. NotificationSetting is keyed by userId with paymentReminders defaulting true. Source: `packages/data/prisma/schema.prisma:1–32`.

The native entity table below records fields and relationships. Server handlers currently return fixture responses rather than visible Prisma queries (inferred from `server/routes/*.ts` and `package.json:10–25`).

Entity	Fields	Relationship / constraint
User	id (String ID), email (unique String), role (String)	1 → many Invoice; 0/1 → NotificationSetting. schema.prisma:9–14.
Invoice	id, ownerId, number, totalMinor (Int), currency, status	ownerId references User.id. schema.prisma:16–24.
NotificationSetting	userId (String ID), paymentReminders (Boolean, default true)	userId references User.id; one optional setting per User. schema.prisma:26–30.

Client DTOs

SessionUser is id/email/role, with role restricted in TypeScript to user or admin. Invoice is id, ownerId, number, integer totalMinor, USD/INR currency, and draft/open/paid status. Source: `src/types.ts:1–16`. The Prisma strings for role, currency, and status are less constrained than the TypeScript unions (inferred schema-contract drift).

Contract gaps to resolve before mobile

- Login request is JSON but the client does not set Content-Type; server uses `express.json()`. Define request headers, validation, password verification, error schema, token expiry, refresh rotation, and revocation. Sources: `src/services/auth.ts:3–7`; `server/index.ts:11`; `server/routes/auth.ts:5–12`.
- Invoice list currently returns `[]`; checkout uses a fixed amount instead of reading a persisted invoice. Define ownership authorization, pagination, currency/amount source, idempotency, result verification/webhooks, and error responses (inferred). Sources: `server/routes/invoices.ts:6–10`; `src/integrations/stripe.ts:5–12`.

- Settings client PUT /api/settings has no matching mount; server exposes GET/PUT /api/notifications/preferences. Select one canonical contract and align client, auth, validation, and persistence. Sources: src/services/settings.ts:3–5; server/routes/notifications.ts:3–5; server/index.ts:16–24.
- Push client POST /api/notifications/installations has no matching mounted route. Define device identifier, platform, token rotation, proof/auth model, deletion, consent, and delivery provider (inferred). Sources: src/services/notifications.ts:5–7; server/index.ts:16–24.
- Manifest routes expose public catalog and authenticated audit-events; define whether mobile consumes either and their DTOs. Source: server/routes/manifest.ts:3–8.
- Admin reports lacks a corresponding API contract; UI-only role gating is insufficient (inferred). Sources: app/admin-reports/page.tsx:4–7; server/routes/manifest.ts:5–8.

Proposed contract discipline (inferred)

Publish an OpenAPI or equivalent versioned schema; generate or validate TypeScript DTOs; distinguish transport DTOs from Prisma entities; add consumer/provider contract tests; use HTTPS environment origins; normalize error codes; and preserve backward compatibility while legacy clients are retired.

Delivery Plan

Delivery Plan

Four phases, Sprint 1–6 sequencing, and acceptance gates

The plan groups work into four Jira Tasks. Sprint spans describe execution windows; a Jira issue can have one current Sprint field, so the earliest/anchor sprint is used where available and the full span remains in its description (inferred planning convention). All issues remain To Do for review.

Dependency order

Foundation and auth → Core mobile screens and payments → Notifications, analytics, and observability → Testing and release readiness. The phases overlap only after their prerequisite contracts and foundations pass the stated gates (inferred).

Sprint map

The native table below maps Sprint 1 through Sprint 6, scope, dependencies, and done gates. It is a draft plan, not a commitment to deploy.

Phase / sprint span	Scope and dependency	Done gate (inferred)
Foundation and auth — Sprint 1–2; anchor Sprint 1	Mobile workspace, environment/HTTPS API strategy, navigation shell, typed contracts, secure storage, production identity, refresh/revocation, role enforcement, legacy retirement decision, secret remediation. No predecessor.	iOS/Android dev builds; login/refresh/logout/401 contract tests; server-side authorization; no bundled secrets; reviewed architecture and environment separation.
Core mobile screens and payments — Sprint 2–4; anchor Sprint 3	Depends on foundation contract gate. Build Dashboard, Invoices, Settings, Admin branch, loading/empty/error/offline states; align settings API; invoice persistence/ownership; Stripe checkout and verified deep-link return.	Core journeys pass on both platforms; settings and invoice contracts align; duplicate payment prevention, success/cancel recovery, role denial, accessibility, and deep-link tests pass.
Notifications, analytics, and observability — Sprint 4–5; anchor Sprint 5	Depends on authenticated core shell and canonical preferences contract. Implement flag, permission/token lifecycle, installation API, preference persistence, consented analytics, mobile crash reporting and alert ownership.	Flag-off/on behavior, token rotation/removal, preference sync, consent/PII controls, event taxonomy, error capture and monitoring tests pass on both platforms.

Phase / sprint span	Scope and dependency	Done gate (inferred)
Testing and release readiness — Sprint 5–6; anchor Sprint 6	Depends on preceding feature gates. Expand unit/component/contract/E2E and real-device matrix; CI signing, staging distribution, privacy/store assets, payment-policy review, performance, accessibility, upgrade/rollback rehearsal.	Signed staging builds, quality gates and release checklist approved; store submissions prepared but not shipped; rollback and ownership documented; Jira remains To Do in this draft.

Cross-phase definition of done (inferred)

- Cutoff-source traceability and reviewed API contracts; no credential-shaped values in mobile, logs, artifacts, or Jira.
- iOS and Android behavior tested for auth lifecycle, role enforcement, deep links, payment cancellation, network failure, permissions, accessibility, and upgrades.
- Environment separation, signing, privacy disclosures, analytics consent, crash reporting, monitoring, rollback, and store-review evidence approved before release.
- No Jira status transition or production deployment is performed by this draft workflow.

Risks & Open Questions

Risks & Open Questions

Build blockers and decisions to resolve

P0 — resolve before foundation sign-off

- Authentication is fixture-grade: password is not visibly verified, role derives from email suffix, subject is fixed, refresh is static/unguarded, and fallback signing/verification material exists. Define production identity, token claims, expiration, rotation, revocation, algorithm/key management, and server-side authorization. Sources: `server/routes/auth.ts:5–12`; `server/middleware/requireAuth.ts:4–13`.
- Credential-shaped material exists in the cutoff tree. Security must assess authenticity, rotate/revoke if necessary, remove from history/config, and enforce scanning. See Secrets found (review); values are intentionally omitted.
- Deployment topology is not established in the committed fixture: Next and Express have separate scripts and the client uses relative API paths. Who owns the HTTPS mobile API origin, gateway, CORS, environments, and versioning? Sources: `package.json:5–8`; `server/index.ts:16–26`; `src/services/*.ts`.

P1 — resolve before core feature completion

- Settings and push-installation client routes do not match mounted server handlers. Choose canonical contracts and implement persistence/validation before mobile UI integration. Sources: `src/services/settings.ts:3–5`; `src/services/notifications.ts:5–7`; `server/index.ts:16–24`; `server/routes/notifications.ts:3–5`.
- Invoice list and checkout are fixture stubs: empty list, fixed amount, no visible ownership lookup or webhook/result reconciliation. Define persistence, authorization, idempotency, currency handling, and checkout-return verification. Sources: `server/routes/invoices.ts:6–10`; `src/integrations/stripe.ts:5–12`; `packages/data/prisma/schema.prisma:14–24`.
- Custom-scheme checkout return has no visible detail/result screen. Decide universal/app links, domain association, scheme ownership, cold-start routing, and cancellation recovery (inferred). Source: `src/integrations/stripe.ts:8–11`.
- Admin page takes a user prop and performs a UI role check; establish actual session injection and server-enforced report authorization. Source: `app/admin-reports/page.tsx:4–7`.

P2 — resolve before beta/release

- Push UI is flag-disabled while preferences are live; installation endpoint is absent. Decide provider, consent copy, APNs/FCM credentials, token retention, notification categories, and rollout. Sources: `app/notifications-center/page.tsx:3–6`; `server/routes/notifications.ts:3–5`; `src/services/notifications.ts:5–7`.

- PostHog browser SDK startup occurs from Express initialization, while invoice event helper has no visible caller; Sentry is off by default. Decide mobile SDKs, privacy consent, event taxonomy, PII controls, environment sampling, and alert ownership. Sources: `server/index.ts:2–13`; `src/integrations/posthog.ts:1–9`; `src/integrations/sentry.ts:3–7`; `.env.example:4`.
- No visible test files, mobile project, CI workflow, accessibility implementation, offline cache, or release configuration exists in the cutoff tree (inferred from the complete file list). Set quality thresholds and owners.
- Determine whether invoice payments are for physical/external services or digital in-app goods, and obtain app-store policy/legal review before selecting Stripe external checkout versus platform billing (inferred). Source for current checkout design: `src/integrations/stripe.ts:5–12`.

Open decisions

Who owns identity and API contracts? Which legacy clients need cookie endpoints and when can they retire? Is catalog/audit-events in mobile scope? What are the minimum iOS/Android versions, supported locales/currencies, offline expectations, accessibility standard, analytics consent model, data-retention policy, release owner, and rollback SLA? (inferred)

Secrets found (review)

Secrets found (review)

Locations only — do not copy values into this document, Jira, logs, or chat

The README identifies credential-shaped fixture material for scanner-review testing. Treat each location as potentially sensitive until security verifies whether it is placeholder, synthetic, or active. No value is reproduced here. Source: README.md:13.

Review locations

- .env.example:1–3 — public-key, payment-provider, and analytics credential-shaped placeholders; confirm placeholders cannot be mistaken for deployable values.
- src/config/cloud.ts:2 — cloud access-key identifier-shaped literal; assess validity and exposure, rotate/revoke if real, remove from source/history as appropriate.
- src/services/notifications.ts:3 — encoded bootstrap/proof-shaped constant marked for removal before launch; assess, revoke/rotate if applicable, and replace with a server-issued design.
- server/routes/auth.ts:8,12 — fallback signing material and fixed refresh-token-shaped response; remove fixture behavior from any deployable environment.
- server/middleware/requireAuth.ts:8 — verification-key fallback; fail closed when production configuration is absent.
- server/auth/legacySession.ts:6 — cookie signing-key fallback; replace with managed secret and review cutoff cookie attributes.
- app/settings/page.tsx:6 — hard-coded session-token-shaped argument; remove from client example and use authenticated session state.
- app/login/page.tsx:6 — hard-coded demo sign-in arguments; remove from production UI and ensure no credential values are bundled.

Required response (inferred)

Security owner should classify each finding, verify exposure without redistributing values, revoke/rotate any real material, purge or rewrite history under an approved incident process if needed, move configuration to managed secret stores, fail closed outside development, add pre-commit and CI secret scanning, and document closure evidence. Do not attach source files containing the values to Jira.